MAANG STAFF ENGINEER INTERVIEW SERIES

JavaScript Event Loop & Concurrency Model

Complete Reference Manual: Theoretical Fundamentals & Execution Traces

Q1. Explain the Call Stack, Heap, and Event Loop in JavaScript. How do they interact?

ANSWER: JavaScript relies on a single-threaded runtime model driven by three primary components:

- **Call Stack:** A LIFO (Last-In, First-Out) data structure that tracks active function execution frames. When a function is invoked, its stack frame is pushed; when it returns, it is popped.
- **Heap:** A large, unstructured memory region where objects, arrays, functions, and dynamic allocations reside.
- **Event Loop:** A continuous, single-threaded loop that monitors the Call Stack and task queues. When the Call Stack becomes completely empty, the Event Loop takes queued callbacks from microtask and macrotask queues and pushes them onto the Call Stack for execution.

Q2. What is the difference between a Macrotask and a Microtask?

ANSWER: Both represent asynchronous work, but microtasks take higher priority and completely drain before any macrotask runs.

Property	Microtask Queue	Task Queue (Macrotask)
APIs	`Promise.then/catch/finally`, `queueMicrotask()`, `MutationObserver`, `process.nextTick`	`setTimeout()`, `setInterval()`, `setImmediate()`, `requestAnimationFrame()`, I/O callbacks
Execution Timing	Immediately after current stack frame empties, before UI rendering or next macrotask.	One callback per event loop iteration after the microtask queue is fully drained.
Queue Draining	Drained completely in a loop (including newly added microtasks during execution).	Only one task is processed per loop tick before returning control to the loop.

Q3. Synchronous vs Asynchronous Execution in JavaScript

ANSWER: **Synchronous code** executes sequentially on the Call Stack; execution blocks the thread until the current operation finishes. **Asynchronous code** offloads operations (such as timers, network I/O, or file operations) to Web APIs or C++ worker threads. When completed, callbacks are queued to be picked up by the Event Loop once the main stack is idle.

Q4. How does the Event Loop decide execution priority between task queues?

ANSWER: The Event Loop follows a strict deterministic algorithm:

1. Execute all synchronous code on the Call Stack until empty.
2. Drain the **Microtask Queue** completely (processing recursively added microtasks).
3. Perform DOM rendering/re-flow if required (browser refresh rate permitting).
4. Pick the oldest callback from the **Macrotask Queue**, push it onto the Call Stack, and run it.
5. Repeat the cycle from Step 2.

Q5. Why do Promise callbacks run before `setTimeout(fn, 0)`?

ANSWER: Promises register callbacks in the **Microtask Queue**, while `setTimeout(fn, 0)` registers its callback in the **Macrotask Queue**. After the synchronous script finishes, the Event Loop checks and fully empties the Microtask Queue before moving to the Macrotask Queue.

Q6. Scheduling API Comparison Matrix

API	Type	Environment	Priority Tier
`process.nextTick()`	NextTick Queue	Node.js	Highest (Runs before standard Microtasks)
`queueMicrotask()`	Microtask	Browser & Node.js	High (Standard Microtask queue)
`Promise.then()`	Microtask	Browser & Node.js	High (Standard Microtask queue)

Q7. What is Event-Loop Starvation? How is it caused and prevented?

ANSWER: **Event-Loop Starvation** occurs when the Event Loop is prevented from processing macrotasks, I/O events, rendering updates, or user interactions.
Cause: Continuously enqueuing microtasks recursively (e.g., a function inside `queueMicrotask` calling `queueMicrotask`). Because the runtime drains the microtask queue completely before yielding control, macrotasks and I/O callbacks are starved indefinitely.
Prevention: Use macrotasks like `setTimeout(fn, 0)` or chunk heavy computation into async iterations using `scheduler.yield()` or web workers.

Q8. How does the Node.js Event Loop differ from the Browser Event Loop?

ANSWER: The browser event loop focuses heavily on DOM rendering, user input, and frame generation. Node.js relies on **libuv** to handle asynchronous I/O, file operations, and networking using system-level event notifications (epoll, kqueue). Node's event loop is divided into distinct execution phases, whereas browsers rely on simple microtask/macrotask queue processing between paint cycles. Node.js also introduces `process.nextTick()`, which executes before microtasks.

Q9. `process.nextTick()` vs `queueMicrotask()` in Node.js

ANSWER:

- `process.nextTick()` places callbacks into Node's specific **nextTickQueue**. It runs immediately after the currently executing operation finishes, before moving to the standard Microtask Queue.
- `queueMicrotask()` uses the cross-platform V8 microtask queue (shared with Promises).
- **Use Case:** Use `process.nextTick()` when an object needs to emit an event or run setup logic right after construction but before I/O continues. Use `queueMicrotask()` for standard cross-environment asynchronous tasks.

Q10. Explain the Phases of the Node.js Event Loop (libuv)

ANSWER: Each loop iteration in libuv processes through explicit phases in order:

Phase	Description & Handled Callbacks
Timers Phase	Executes callbacks scheduled by `setTimeout()` and `setInterval()` whose threshold elapsed.
Pending Callbacks	Executes I/O callbacks deferred from previous iterations (e.g., TCP errors).
Idle, Prepare	Internal libuv housekeeping routines.
Poll Phase	Retrieves new I/O events (file system, socket reading). Blocks here if no timers are pending.
Check Phase	Executes callbacks scheduled specifically by `setImmediate()`.
Close Callbacks	Executes cleanup handlers (e.g., `socket.on('close')`).

Q11. `setTimeout` vs `setInterval` & Recursive `setTimeout`

ANSWER: `setInterval` schedules execution at fixed time intervals regardless of how long the callback takes to execute, potentially causing task stacking if callback duration exceeds interval time. **Recursive `setTimeout`** guarantees a fixed delay *after* the callback execution completes, preventing concurrent execution and drift.

Q12. How does long-running synchronous JS affect UI and Rendering?

ANSWER: Browsers operate on a rendering pipeline targeting 60fps (~16.6ms frame budget). Long-running synchronous code blocks the Call Stack, preventing the event loop from reaching the rendering phase. This leads to dropped frames, frozen animations, and non-responsive user input ("jank"). Solutions include offloading to Web Workers or splitting CPU-heavy work across animation frames.

Q13. Basic Event Loop Execution Order

```
console.log("A"); setTimeout(() => console.log("B"), 0); Promise.resolve().then(() => console.log("C")); console.log("D");
```

OUTPUT: `A -> D -> C -> B`

Step-by-Step Trace:

1. Synchronous: `console.log("A")` executes -> Prints **A**.
2. `setTimeout` registers callback in Macrotask queue.
3. `Promise.resolve().then` registers callback in Microtask queue.
4. Synchronous: `console.log("D")` executes -> Prints **D**.
5. Call Stack empties. Event Loop drains Microtask queue -> Prints **C**.
6. Microtask queue empty. Event Loop processes Macrotask queue -> Prints **B**.

Q14. Nested Microtask Scheduling

```
Promise.resolve().then(() => { console.log("A"); Promise.resolve().then(() => console.log("B")); }); Promise.resolve().then(() => console.log("C")); console.log("D");
```

OUTPUT: `D -> A -> C -> B`

Step-by-Step Trace:

1. P1 registered (Microtask 1), P2 registered (Microtask 2). Sync `console.log("D")` -> Prints **D**.
2. Microtask 1 runs: prints **A**, enqueues inner promise P3 into Microtask queue.
3. Microtask 2 runs: prints **C**.
4. Microtask 3 (newly enqueued P3) runs: prints **B**.

Q15. `queueMicrotask()` Interacting with Promises

```
console.log("Start"); queueMicrotask(() => console.log("A")); Promise.resolve().then(() => console.log("B")); queueMicrotask(() => console.log("C")); console.log("End");
```

OUTPUT: `Start -> End -> A -> B -> C`

Step-by-Step Trace: Synchronous logs **Start** and **End** execute first. Microtasks are queued in order: `queueMicrotask(A)`, `Promise(B)`, `queueMicrotask(C)`. All microtasks belong to the same microtask queue and execute FIFO: **A**, **B**, **C**.

Q16. Nested Timers & Microtask Checkpoints

```
console.log("A"); setTimeout(() => { console.log("B"); setTimeout(() => console.log("C"), 0); Promise.resolve().then(() => console.log("D")); }, 0); setTimeout(() => console.log("E"), 0); console.log("F");
```

OUTPUT: `A -> F -> B -> D -> E -> C`

Step-by-Step Trace: Sync logs **A**, **F**. Macrotask queue: [Timer B, Timer E]. Timer B runs: prints **B**, enqueues Timer C (Macrotask) and Promise D (Microtask). Stack empties -> Microtask checkpoint runs: prints **D**. Next Macrotask Timer E runs -> prints **E**. Finally, Timer C runs -> prints **C**.

Q17. Microtask Checkpoints between Tasks

```
setTimeout(() => console.log("A"), 0); Promise.resolve().then(() => { console.log("B"); queueMicrotask(() => console.log("C")); }); Promise.resolve().then(() => console.log("D")); setTimeout(() => console.log("E"), 0);
```

Q19. Async / Await Microtask Mechanics

```
async function run() { console.log("A"); await Promise.resolve(); console.log("B"); }
console.log("C"); run(); console.log("D");
```

OUTPUT: `C -> A -> D -> B`

Step-by-Step Trace: Sync prints **C**. `run()` called synchronously: prints **A**. At `await`, execution pauses and the remainder of `run()` is scheduled as a Microtask. Call stack pops back to top level: prints **D**. Sync script finishes. Event loop drains Microtask queue: prints **B**.

Q20. Promise Chaining vs Timer Execution

```
setTimeout(() => console.log("Timer"), 0); Promise.resolve() .then(() => { console.log
("Promise 1"); }) .then(() => { console.log("Promise 2"); }); console.log("Sync");
```

OUTPUT: `Sync -> Promise 1 -> Promise 2 -> Timer`

Step-by-Step Trace: Prints **Sync**. Timer queued as Macrotask. Promise chain tick 1 executes -> prints **Promise 1** and returns, which immediately enqueues the second `.then()` into the Microtask queue. The microtask queue is fully drained before taking macrotasks -> prints **Promise 2**. Finally Macrotask runs -> prints **Timer**.

Q21. Node.js `process.nextTick()` Priority

```
console.log("Start"); process.nextTick(() => console.log("NextTick")); Promise.resolve
().then(() => console.log("Promise")); queueMicrotask(() => console.log("Microtask")); consol
e.log("End");
```

OUTPUT (NODE.JS): `Start -> End -> NextTick -> Promise -> Microtask`

Step-by-Step Trace: Sync logs **Start**, **End**. Node's Event Loop checks `nextTick`Queue first before standard Microtask queue -> prints **NextTick**. Then standard Microtasks execute FIFO -> **Promise**, **Microtask**.

Q22. Node.js I/O Callback Phase Context (CommonJS vs ESM)

```
const fs = require("node:fs"); fs.readFile(__filename, () => { setTimeout(() => consol
e.log("Timeout"), 0); setImmediate(() => console.log("Immediate")); });
```

OUTPUT (INSIDE I/O CALLBACK): `Immediate -> Timeout`

Explanation: Inside an I/O callback, execution occurs in libuv's **Poll Phase**. When the callback completes, the event loop moves immediately to the **Check Phase** (where `setImmediate` callbacks reside) before looping back around to the **Timers Phase**. In ESM modules, top-level module wrapping differs, but inside I/O callbacks, `setImmediate` consistently runs before `setTimeout`.

Q23. Non-deterministic Top-Level `setTimeout` vs `setImmediate`

```
setTimeout(() => console.log("Timeout"), 0); setImmediate(() => console.log("Immediat
e"));
```

OUTPUT: `Non-Deterministic (Timeout -> Immediate OR Immediate -> Timeout)`

Explanation: At top-level script scope, timing depends on system performance and loop entry overhead. If process boot takes <1ms, timer threshold isn't met yet (Immediate runs first). If boot takes >1ms, timer threshold is passed (Timeout runs first).

Q24. Promise Microtask Cross-Scheduling

Q25. Synchronous Loop Blocking Timer Execution

```
setTimeout(() => console.log("Timer"), 0); const start = Date.now(); while (Date.now() - start < 100) { /* Block thread for 100ms */ } console.log("Loop finished");
```

OUTPUT: `Loop finished -> Timer`

Explanation: Synchronous code occupies the main Call Stack. Even though the 0ms timer threshold expires almost immediately, the Event Loop cannot process the Macrotask queue until the Call Stack is completely clear.

Q26. Continuous Microtask Scheduling & Infinite Loops

```
function repeat() { queueMicrotask(repeat); } repeat(); setTimeout(() => console.log("Timer"), 0);
```

RESULT: `Main Thread Freeze / Event Loop Starvation`

Explanation: `repeat()` enqueues a new microtask every time it runs. Because the event loop must completely empty the microtask queue before picking up any macrotasks, the timer callback will **never execute** and the browser UI will freeze.

Q27. Synchronous Stack Call Execution Order

```
function one() { console.log("One"); two(); console.log("Three"); } function two() { console.log("Two"); three(); } function three() { console.log("Four"); } one(); console.log("Five");
```

OUTPUT: `One -> Two -> Four -> Three -> Five`

Q28. Promise Chaining & Resolution Overhead

```
Promise.resolve() .then(() => { console.log("A"); return "B"; }) .then((value) => { console.log(value); return Promise.resolve("C"); }) .then((value) => console.log(value)); console.log("D");
```

OUTPUT: `D -> A -> B -> C`

Deep Dive Trace: Synchronous log prints **D**. Chain tick 1 runs: prints **A**, returns string "B". Chain tick 2 runs: prints **B**, returns `Promise.resolve("C")`. When returning a resolved Promise from a `.then()` handler, standard ECMAScript specification dictates a 2-microtask-tick unwrapping penalty before downstream handlers trigger.

Key Takeaways for Code Output Questions

- Always execute synchronous code top to bottom first.
- Drain the microtask queue completely before picking up ONE macrotask.
- Remember that `process.nextTick` runs before all other microtasks in Node.js.
- Inside libuv I/O callbacks, `setImmediate` always beats `setTimeout(fn, 0)`.

Q29. CPU-Intensive Tasks in Node.js Request Handlers

PROBLEM ANALYSIS & ARCHITECTURE FIXES:

Performing synchronous CPU-bound operations (such as crypto hashing, image processing, or heavy mathematical loops) blocks the main thread. This starves incoming HTTP requests, delays timers, and halts I/O callbacks across the entire server instance.

Architectural Patterns for Mitigation:

1. **Worker Threads (`worker_threads`):** Offload heavy calculations to real OS-level threads sharing memory via `ArrayBuffer` or `SharedArrayBuffer`.
2. **Child Processes (`child_process.fork()`):** Isolate heavy computation into independent Node processes communicating over IPC.
3. **External Job Queue Architecture:** Offload task parameters to Redis / RabbitMQ and let dedicated worker nodes process calculations asynchronously.

Q30. System Design: Production Async Job Queue with Exponential Backoff

SYSTEM ARCHITECTURE & CORE MECHANICS:

Designing an in-memory or distributed job processing system that executes asynchronous tasks reliably without stalling the runtime event loop requires careful concurrency bounds, retry strategies, and idempotency guarantees.

```
class AsyncJobQueue { constructor(concurrency = 5, maxRetries = 3) { this.concurrency = concurrency; this.maxRetries = maxRetries; this.running = 0; this.queue = []; } enqueue(jobFn, id) { return new Promise((resolve, reject) => { this.queue.push({ jobFn, id, attempts: 0, resolve, reject }); this.processNext(); }); } async processNext() { if (this.running >= this.concurrency || this.queue.length === 0) return; this.running++; const job = this.queue.shift(); try { const result = await job.jobFn(); job.resolve(result); } catch (err) { if (job.attempts < this.maxRetries) { job.attempts++; const backoffDelay = Math.pow(2, job.attempts) * 100; // Exponential backoff setTimeout(() => { this.queue.push(job); this.processNext(); }, backoffDelay); } else { job.reject(new Error(`Job ${job.id} failed after ${this.maxRetries} retries: ${err.message}`)); } } finally { this.running--; this.processNext(); } } }
```

Design Key Points:

- **Concurrency Limiting:** Guards against exceeding file descriptor bounds or exhausting system memory.
- **Non-Blocking Timers:** `setTimeout` with exponential backoff schedules retries without blocking the main event loop.
- **Idempotency & Dead Letter Queue (DLQ):** Ensures duplicate triggers do not cause corrupt state, routing permanently failed jobs to a DLQ for inspection.